



DevOps Interview 8/90

Q: Why Git? What problems does Git solve in software development and DevOps?

Intro / Context:

In software development, especially in team environments, managing code changes efficiently is crucial. Before Git, developers struggled with version conflicts, collaboration bottlenecks, and loss of historical changes. Tools like SVN or manual backups were either centralized or error-prone.

Answer:

Git is a distributed version control system that allows developers to track changes in source code, collaborate efficiently, and manage multiple versions of codebases seamlessly.

Problems Git Solves:

1. Version Control:

Git keeps a complete history of every change made to a project, allowing you to revert or compare versions easily.

2. Collaboration:

Multiple developers can work on the same codebase at the same time using branches without overwriting each other's work.

3. Backup and Distribution:

Every developer has a full copy of the repo locally, making Git distributed and fault-tolerant.

4. Branching and Merging:

Developers can create isolated branches for features or bug fixes and merge them without affecting the main codebase.

5. Audit and Blame:

Git helps identify who made what change and why (using `git blame`, `git log`).

6. Automation Friendly:

Git integrates seamlessly with DevOps tools like Jenkins, GitHub Actions, and GitLab CI for automation and CI/CD pipelines.

Bonus for DevOps Interviews:

- **DevOps Lifecycle Integration:**

Git is foundational in the DevOps lifecycle. It acts as the *Single Source of Truth* for code, config, and IaC (Infrastructure as Code).

- **GitOps:**

Modern DevOps teams use Git to manage not just code, but infrastructure and application state through GitOps practices.

Q: What are the basic Git commands DevOps engineer should know, and what do they do?

Introduction to Git:

Git is a **distributed version control system (VCS)** used to track changes in source code. It helps teams collaborate by maintaining a history of all modifications and allowing branching and merging for parallel development.

Key Concepts:

- **Repository (repo):** A project directory tracked by Git.
 - **Working Directory:** Where you edit files.
 - **Staging Area:** Where changes are marked to be committed.
 - **Commit:** A snapshot of the staged changes.
 - **Remote:** A version of your repo hosted on a server like GitHub.
-

Basic Git Commands:

1. Initialize a Repo

```
git init
```

Creates a new local Git repository.

2. Clone a Remote Repo

```
git clone https://github.com/user/repo.git
```

Copies a remote repository to your local machine.

3. Check Repo Status

```
git status
```

Shows modified files and their stages.

4. Track a New File

```
git add filename
```

Adds a file to the staging area.

5. Track All Changes

```
git add .
```

Stages all new and modified files.

6. Commit Changes

```
git commit -m "Message about what you changed"
```

Saves a snapshot of staged changes.

7. Check Commit History

```
git log
```

Shows the commit history.

8. Create a New Branch

```
git branch feature-login
```

9. Switch to a Branch

```
git checkout feature-login
```

10. Merge Branch into Main

```
git checkout main  
git merge feature-login
```

11. Push to Remote Repo

```
git push origin main
```

12. Pull Changes from Remote

```
git pull
```

Real-World Flow Example:

```
git clone https://github.com/MukundLokhande/Product-Category-Manager.  
git  
cd Product-Category-Manager  
touch new-feature.txt  
git add .  
git commit -m "Added new feature file"  
git push origin main
```

Bonus Tip:

In DevOps, Git is also used to:

- Store Infrastructure as Code (like Terraform)
- Manage Kubernetes manifests
- Trigger CI/CD pipelines

Q: What is the difference between a centralized version control system and a distributed version control system like Git? Why is Git preferred in modern DevOps workflows?

Introduction:

Version control systems (VCS) help teams manage and track changes to source code over time. They fall into two main types:

1. Centralized Version Control System (CVCS):

- **Examples:** SVN, CVS, Perforce
- **How it works:** There's a **single central server** that holds the complete project. Developers pull (checkout) files and commit changes **directly to this central server**.

Limitations:

- Requires **constant internet access** to commit changes.
 - If the server crashes or is corrupted, **all history may be lost**.
 - **Slower** collaboration, especially in remote/distributed teams.
-

2. Distributed Version Control System (DVCS):

- **Examples:** Git, Mercurial, Bazaar
- **How it works:** Every developer has a **full copy** (clone) of the repository including full history. Changes can be committed **locally** and pushed to a shared remote (like GitHub) when ready.

Advantages:

- Works **offline** – you can commit without internet.
 - Full backup of history on every developer's machine.
 - **Branching and merging** is easier and faster.
 - Better suited for **collaboration and CI/CD pipelines**.
-

Key Differences at a Glance:

Conclusion:

Git, as a distributed system, offers **flexibility, reliability, and robust collaboration**, making it the preferred choice in **DevOps** and **modern software development workflows**.

Q: What are some commonly used Git commands in daily DevOps workflows? Can you explain them with real examples?

Intro:

Git is the backbone of version control in DevOps. Understanding key Git commands helps teams manage source code, collaborate effectively, and automate deployments.

Answer with Examples:

Here's a curated list of essential Git commands and how they're used:

1. `git init`

Description: Initializes a new Git repository in your current project directory.

Example:

```
mkdir demo-project && cd demo-project  
git init # Now this folder is tracked by Git
```

2. `git clone`

Description: Creates a local copy of a remote repository.

Example:

```
git clone https://github.com/MukundLokhande/Product-Category-Manager.  
git
```

3. `git status`

Description: Shows the status of your working directory and staging area.

Example:

```
git status
```

```
# Tells you which files are changed or untracked
```

4. **git add**

Description: Stages changes to be committed.

Example:

```
git add file1.py # stage a specific file
```

```
git add . # stage all changes
```

5. **git commit**

Description: Saves the staged changes to your local repository.

Example:

```
git commit -m "Add login feature"
```

6. **git push**

Description: Sends your commits to the remote repository.

Example:

```
git push origin main
```

7. **git pull**

Description: Fetches and merges updates from the remote repo.

Example:

```
git pull origin main
```

8. **git branch**

Description: Lists, creates, or deletes branches.

Example:

```
git branch feature-xyz # create  
git checkout feature-xyz # switch to branch
```

9. **git merge**

Description: Merges changes from one branch into another.

Example:

```
git checkout main  
git merge feature-xyz
```

10. **git log**

Description: Shows commit history.

Example:

```
git log --oneline
```

Why it Matters in DevOps:

- Helps **track code changes** across features and environments.
- Essential for **CI/CD automation**.
- Enables **team collaboration** and **clean code integration**.